

Cientes SSH y autenticación con clave pública

Práctica 1

DESPLIEGUE DE APLICACIONES WEB

1. El objetivo de la presente práctica es profundizar en el conocimiento acerca de las distintas soluciones cliente que puedan existir para los sistemas operativos más utilizados (GNU/Linux, Windows, Mac OSX, iOS y Android).

Para ello, debes realizar búsquedas en la web y localizar información de al menos 2 clientes SSH para cada sistema operativo nombrados, y que hayan sido actualizados en los últimos dos años, es decir, no incluir proyectos inactivos.

Debes construir una tabla en formato apaisado que contenga por filas el nombre de la aplicación y por columnas los siguientes registros:

- Desarrollador
- Año de lanzamiento
- Última actualización
- Última versión estable
- Tipo de licencia
- Sistema(s) operativo(s) bajo lo(s) que puede ser instalada la aplicación
- Web

He realizado una búsqueda exhaustiva para identificar clientes SSH que cumplan con los requisitos de mantenimiento (actualizados en los últimos dos años) y versatilidad. A continuación, presento la tabla comparativa con los hallazgos:

Aplicación	Desarrollador	Lanzamiento	Última Actualización	Versión Estable	Licencia	SO compatibles	Web
PuTTY	Simon Tatham	1999	2024	0.81	MIT	Win, Linux, Mac	putty.org
MobaXterm	Mobatek	2008	2025	24.3	Propietaria	Windows	mobaxterm.mobatek.net
Termius	Termius Corp	2013	2026	9.x	Propietaria	Multiplataforma	termius.com
OpenSSH	OpenBSD	1999	2026	9.9p1	BSD	Win, Linux, Mac	openssh.com
Blink Shell	Blink	2016	2026	17.x	GPLv3	iOS, iPadOS	blink.sh
ConnectBot	Kenny Root	2007	2025	1.9.8	Apache 2.0	Android	connectbot.org

- Realiza la configuración del **contenedor servidor SSH** con Ubuntu y conectate desde otro **contenedor SSH cliente**, de manera que puedas

hacerlo usando autenticación con clave pública, de modo que no se necesite proporcionar los datos de acceso (contraseña) para establecer la conexión en adelante.

- a. Busca información sobre las claves públicas y privadas autogeneradas.
- b. Busca información sobre el comando `ssh-keygen` y sus opciones más importantes.

Ayuda:

1. En el cliente, generamos las claves pública/privada desde el directorio **.ssh** del usuario:

```
$ssh-keygen -t rsa -b 2048
```

- 2a. Copiar (`scp`) la clave pública al home del usuario en el **servidor** en:

```
~/.ssh/authorized_keys.
```

- 2b. También se puede copiar la clave pública usando:

```
$ssh-copy-id -i ~/.ssh/id_rsa.pub IP-remote-host
```

3. Establecer de nuevo la conexión usando:

```
$ssh IP-remote-host
```

Crear el Servidor:

```
root@6274f076841d: /
root@6274f076841d: / 80x24
alumnado@iespsur:~$ docker run -d --name servidor-ssh ubuntu:latest sleep infinity
6274f076841d93d302ffcc2f787f2d2499093ae8b02ea39e484589f33c2aa7a9
alumnado@iespsur:~$ docker exec -it servidor-ssh bash
root@6274f076841d:/# apt update && apt install -y openssh-server
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3491 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1498 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1985 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [34.2 kB]
51% [6 Packages 9852 kB/19.3 MB 51%]
```

Se crea el contenedor llamado cliente-ssh. El parámetro -d permite que corra en segundo plano, y sleep infinity se utiliza para evitar que el contenedor se detenga inmediatamente al no tener un proceso interactivo inicial. Esto nos proporciona un sistema operativo limpio para empezar a trabajar.

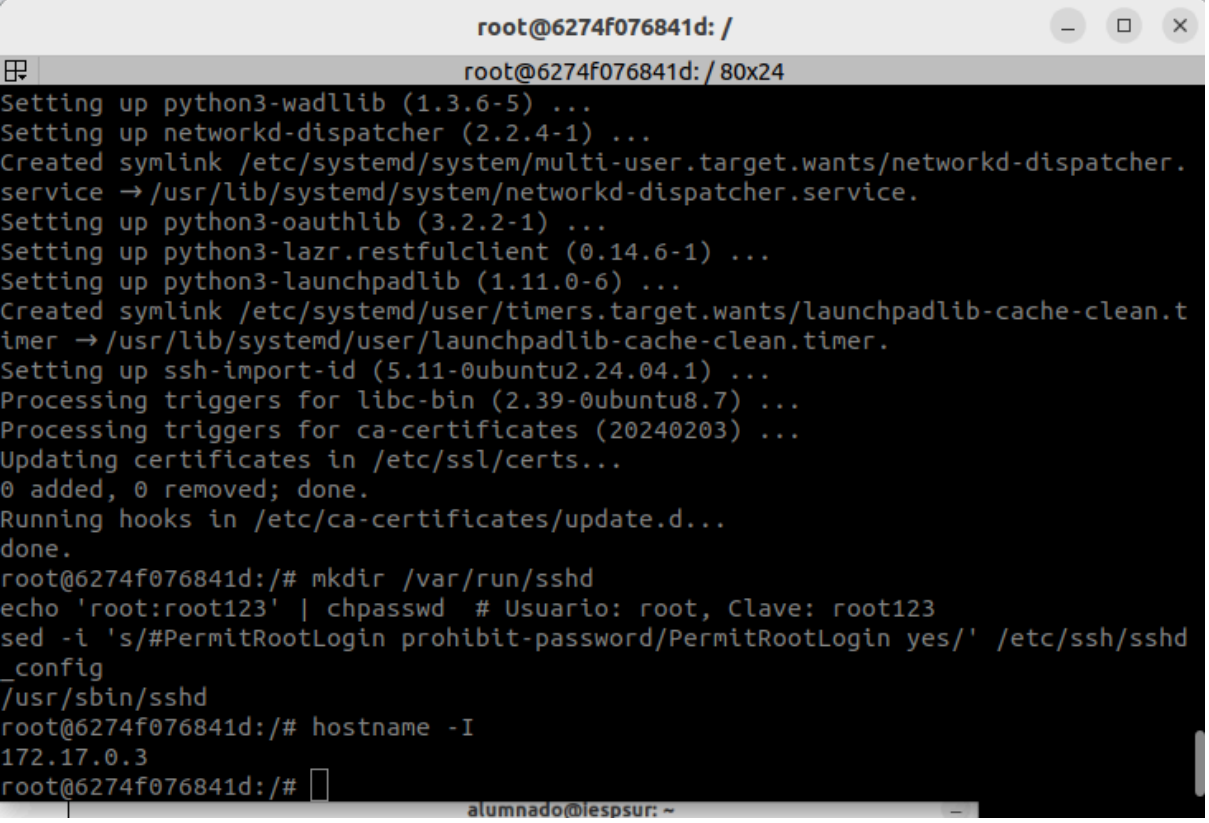
```
alumnado@iespsur: ~
alumnado@iespsur: ~ 80x21
alumnado@iespsur:~$ docker run -d --name cliente-ssh ubuntu:latest sleep infinity
15cff81bb01f1e7b63ed397ef5b886aa8da19547babda8e672fb712ea2ad68a7
alumnado@iespsur:~$
```

Instalación: Se instalan los paquetes necesarios para el servicio SSH.

Seguridad: Se crea el directorio de ejecución /var/run/sshd y se establece una contraseña temporal para el usuario root (root123).

Permisos: Se modifica el archivo sshd_config para permitir el acceso remoto al usuario root.

Identificación: Finalmente, el comando hostname -I nos revela la IP del servidor (172.17.0.3), que será el destino de nuestras conexiones.



```
root@6274f076841d: /  
root@6274f076841d: / 80x24  
Setting up python3-wadllib (1.3.6-5) ...  
Setting up networkd-dispatcher (2.2.4-1) ...  
Created symlink /etc/systemd/system/multi-user.target.wants/networkd-dispatcher.service → /usr/lib/systemd/system/networkd-dispatcher.service.  
Setting up python3-oauthlib (3.2.2-1) ...  
Setting up python3-lazr.restfulclient (0.14.6-1) ...  
Setting up python3-launchpadlib (1.11.0-6) ...  
Created symlink /etc/systemd/user/timers.target.wants/launchpadlib-cache-clean.timer → /usr/lib/systemd/user/launchpadlib-cache-clean.timer.  
Setting up ssh-import-id (5.11-0ubuntu2.24.04.1) ...  
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...  
Processing triggers for ca-certificates (20240203) ...  
Updating certificates in /etc/ssl/certs...  
0 added, 0 removed; done.  
Running hooks in /etc/ca-certificates/update.d...  
done.  
root@6274f076841d:/# mkdir /var/run/sshd  
echo 'root:root123' | chpasswd # Usuario: root, Clave: root123  
sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config  
/usr/sbin/sshd  
root@6274f076841d:/# hostname -I  
172.17.0.3  
root@6274f076841d:/#
```

Desde el contenedor cliente, enviamos nuestra clave pública al servidor.

Al ejecutar `ssh-copy-id root@172.17.0.3`, el sistema nos pide la contraseña del servidor por última vez.

Una vez introducida, el cliente copia el contenido de su `id_rsa.pub` dentro del archivo `~/.ssh/authorized_keys` del servidor. A partir de este momento, el servidor "reconoce" la firma digital del cliente.

```
root@15cff81bb01f: /
root@15cff81bb01f: / 80x21
refused

root@15cff81bb01f:/# ssh-copy-id root@172.17.0.3
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
The authenticity of host '172.17.0.3 (172.17.0.3)' can't be established.
ED25519 key fingerprint is SHA256:5zDXsEwB80WAHXBzukGhB/UcVMSuVVajmWCGIB9cwHY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
root@172.17.0.3's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'root@172.17.0.3'"
and check to make sure that only the key(s) you wanted were added.

root@15cff81bb01f:/#
```

Se ejecuta el comando `ssh root@172.17.0.3`. Como se puede observar, el sistema no solicita contraseña y muestra directamente el banner de bienvenida de Ubuntu dentro del servidor. El prompt cambia de `root@15cff...` (cliente) a `root@6274f...` (servidor), confirmando que hemos establecido una sesión segura basada exclusivamente en el par de claves RSA generadas previamente.

```
root@15cff81bb01f:/# ssh root@172.17.0.3
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-110-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
root@6274f076841d:~#
```